Historico do Bluetooth (removido do firmware)

Atualizacao atual: Prometheus sem Bluetooth

Esta atualizacao nao reativa Bluetooth. A configuracao Prometheus usa `/prometheus` no painel HTTP da **porta 80** e Preferences `telemetry`. O servidor coleta `/metrics` na porta configurada e salva (padrao **9120**); o endereco do servidor e uma referencia, nao envio ativo nem autorizacao. O YAML baixado deve ser aplicado no servidor. Serial USB e UDP permanecem inalterados.

Foram acrescentadas metricas reais de sistema, Wi-Fi, CPU, memoria, GPIO, HTTP, temperatura interna, idade do clima e cartao SD. Capacidade, ocupacao, montagem, frequencia e contadores de log do SD usam cache sem novas operacoes SPI por coleta. Metricas sem integracao real permanecem omitidas. Consulte PROMETHEUS.md para configuracao e testes; as secoes de Bluetooth abaixo continuam sendo apenas historico, nao instrucoes atuais.

Bluetooth removido a pedido do usuario. Este documento conserva o historico de investigacao e integracao do testador, mas as instrucoes abaixo descrevem versoes antigas. A comunicacao atual usa serial USB em 115200; consulte `SERIAL_USB.md`. Nao ha mais inicializacao, filas ou pareamento Bluetooth.

Protecao de RAM: depois de montar o SD, a Central exige 80 KB livres para continuar iniciando Wi-Fi com Bluetooth. Depois da conexao, exige 32 KB para as tarefas e consultas. Abaixo desses limites, encerra SPP, host e controlador e libera suas filas. O log informa que Bluetooth foi desligado por falta de RAM; essa recuperacao prioriza touch, USB, UDP, web e OTA e nao confirma que Bluetooth possa permanecer ativo nesta combinacao de recursos. A tarefa de rede tem uma nova tentativa apos essa liberacao se sua criacao falhar. O console usa L2CAP basico e nao consulta mais o core dump automaticamente ao conectar; use `crash_info` explicitamente.

`crash_info` le o resumo salvo, grava um arquivo separado no SD como `/crash_20261014_103015.txt`, reabre e confere todo o conteudo, e somente entao apaga o core dump interno. A resposta informa o caminho e o resultado da exclusao. Sem SD ou com falha de escrita/conferencia, preserva o dump. Se ja existir arquivo no mesmo segundo, acrescenta um numero ao nome; sem relogio sincronizado usa `SEM_NTP` e uptime, sem inventar data e hora. Consultas sem dump valido ou com falha de leitura nao apagam dados. A mensagem automatica ao conectar Bluetooth apenas consulta o resumo, sem apaga-lo. O motivo do ultimo reset e o historico em `/log.txt` permanecem disponiveis.

Estado atual: Bluetooth reativado com inicializacao antecipada e protecao. Foram registrados resets com asserts `fixed_queue_enqueue` e `btm_ble_init`. O Bluetooth agora inicia antes do display, SD, Wi-Fi e tarefas da aplicacao. Preserva a memoria do host e libera somente o controlador BLE antes de iniciar o controlador Classic, sem usar `btStartMode` do core 3.3.12. Se houver panic/watchdog antes de 30 segundos de processamento normal, a proxima inicializacao desativa o Bluetooth e preserva USB, Wi-Fi e OTA. O aviso `[BT] Recuperacao` identifica esse modo. Desligar e ligar a alimentacao permite uma nova tentativa. A estabilidade e o pareamento precisam ser validados na placa; a compilacao so verifica o software.

A protecao considera a compilacao: uma nova versao pode tentar iniciar o Bluetooth sem herdar o bloqueio de uma versao anterior. Os resultados do scan Wi-Fi sao liberados antes da associacao; o servidor aceita um computador, sem canais de audio, e usa filas menores. A tarefa de rede reserva 8 KB de pilha e o Wi-Fi mantem modem sleep para coexistencia com Bluetooth.

Com a Central conectada ao roteador, a pagina de configuracao usa o IP local mostrado em `info`. `192.168.4.1` pertence somente a rede propria `ESP32_CENTRAL_CONFIG`, aberta quando nao consegue conectar a uma rede salva. O clima de Porto Alegre usa HTTP para dados publicos da Open-Meteo, sem credenciais, reduzindo o consumo de RAM com Bluetooth ativo. O quadro de diagnostico seleciona uma profundidade de cor menor quando ha pouca memoria, preservando espaco para rede e consultas. A serial mostra `[CLIMA]` a cada tentativa, com o resultado.

O firmware anuncia **Central-ESP32**, usando Bluetooth classico SPP do ESP32. O computador precisa de Bluetooth e de um terminal serial que suporte SPP. Na Arduino IDE, use **ESP32 Dev Module**, **PSRAM: Disabled** e a particao **Minimal SPIFFS**. Habilitar PSRAM numa placa sem essa memoria adiciona opcoes de compilacao para contornar problemas de cache; nesta combinacao de bibliotecas pode faltar IRAM. IRAM e memoria interna de execucao, e trocar a particao da Flash nao resolve esse erro. O arquivo `build_opt.h` deve acompanhar o sketch. Ele fixa os mesmos pinos e clocks do display usados nesta placa, carrega somente a fonte GLCD que as telas utilizam e otimiza o codigo para tamanho. A configuracao e local a este projeto e e aplicada automaticamente pela Arduino IDE. O Bluetooth serial usa o servidor SPP do SDK com filas limitadas para manter a interface ativa. O firmware permanece compativel com a particao Minimal SPIFFS e com OTA; nao e necessario trocar a tabela de particoes para esta versao. No Windows, adicione Central-ESP32 nos dispositivos Bluetooth e abra a porta COM de saida criada para ele no seu terminal serial. A taxa da porta virtual nao determina a velocidade do Bluetooth. Envie comandos terminados por Enter (CR, LF ou CRLF), com ate 255 bytes. Se o Windows solicitar um PIN legado, use `1234`.

Conectar pelo testador de comandos

Execute testador_comandos.py com Python e a dependencia `pyserial` instalada (`python -m pip install pyserial`). No Windows, primeiro pareie **Central-ESP32** nas configuracoes Bluetooth e consulte a porta COM **de saida** atribuida ao dispositivo. No testador, escolha **Bluetooth**, pressione **Atualizar portas**, selecione ou digite essa COM e pressione **Conectar serial / BT**. Envie `help`, `info`, `reason` ou `crash_info`; as respostas aparecem no painel do testador. O envio tambem conecta automaticamente, mantendo a porta aberta entre comandos. Use **Desconectar** antes de abrir essa porta em outro programa.

Esse modo usa Bluetooth classico SPP, nao BLE, e nao faz descoberta ou pareamento pelo testador. O baud informado e apenas uma configuracao da porta virtual. No Linux, vincule o dispositivo a uma porta RFCOMM pelo sistema e digite, por exemplo, `/dev/rfcomm0`.

Ao conectar, a Central informa o motivo do ultimo reset e o resumo do core dump, quando houver um registro valido. Comandos uteis:

```
help
reason
crash_info
info
sd_log
```

Todos os comandos locais podem ser enviados pelo Bluetooth, incluindo SD, configuracao, reinicio e desligamento. Comandos e respostas sao registrados no SD quando disponivel. USB e Bluetooth possuem buffers de entrada separados. O Bluetooth continua atendendo sem Wi-Fi e pausa durante a gravacao OTA.

O Bluetooth desconecta durante um crash e nao retransmite a mensagem de panic da UART em tempo real. Depois do reinicio, reconecte e envie `crash_info`. Esse comando le o core dump salvo pelo ESP-IDF na particao coredump da Flash, independentemente do SD, e mostra tarefa, motivo, PC, backtrace e SHA256 do ELF. Nem todo crash consegue gravar um dump valido. O registro pode ser antigo e nao e apagado automaticamente. A particao Minimal SPIFFS atual inclui coredump.

Guarde o arquivo `build/Central.ino.elf` correspondente ao firmware gravado: ele permite converter os enderecos do backtrace em funcoes e linhas do codigo. Uma atualizacao muda o ELF; o SHA256 do relatorio identifica a versao da falha.